

Advanced Optimization Passes for the Chiron Intermediate Representation

Your Name / Team Name
CS335 Compiler Design

April 24, 2026

Abstract

This project extends the baseline Chiron framework with a multi-pass optimizer for the Chiron intermediate representation. Starting from the original parsing, IR generation, and CFG infrastructure, we implemented an optimization pipeline based on classical compiler analyses and transformations. The project adds SSA-style metadata construction, Sparse Conditional Constant Propagation, Aggressive Dead Code Elimination, Loop-Invariant Code Motion, CFG simplification, induction-variable optimization, local GVN/CSE, and interval-based range analysis. These passes are integrated through individual command-line flags as well as a combined `-opt-all` mode.

A key design goal was to compose multiple optimizations safely. Since several passes rewrite the IR, the implementation rebuilds CFG and SSA information at important boundaries to avoid using stale analysis results. The transformations are also deliberately conservative: loop optimizations are restricted to safe canonical cases, GVN/CSE is local rather than fully global, and interval analysis only rewrites conditions proven definitely true or definitely false. This allows the optimizer to remove dead assignments, simplify constant and range-provable branches, hoist loop-invariant computations, reduce selected induction-variable computations, and eliminate repeated local expressions while preserving program semantics. The final implementation was validated using targeted tests for individual passes and combined regression tests for the full optimization pipeline.

1 Introduction

Compiler optimizations are an important part of translating programs into a more efficient intermediate form before execution or code generation. Even when the source language is small, the generated intermediate representation can contain redundant computations, unnecessary assignments, constant branches, unreachable blocks, and repeated loop computations. Optimizing this representation can make the final program simpler, easier to analyze, and potentially faster to execute.

This project builds on the baseline Chiron framework, originally provided as the course framework, and extends it with a multi-pass optimization pipeline. The original framework already supported parsing Chiron programs, constructing an intermediate representation, and executing or analyzing that representation. Our work focuses on improving the quality of the generated Chiron IR by adding classical compiler analyses and transformations over it.

The main goal of the project was not just to implement isolated optimizations, but to design a coherent pipeline where multiple passes can expose opportunities for each other. For example, constant propagation can convert a branch condition into a constant, CFG simplification can remove the unreachable branch, and dead-code elimination can then remove assignments that are no longer useful. Similarly, loop and value-based optimizations such as Loop-Invariant Code Motion, induction-variable optimization, and GVN/CSE make the IR cleaner by removing

repeated work, while interval analysis can simplify branches even when values are not known exactly as constants.

A major engineering challenge in the project was that many passes rewrite the IR. Once the IR changes, previously computed CFG or SSA information may become stale. To handle this, the optimizer rebuilds CFG and SSA metadata at important boundaries before running later analyses. This makes the pipeline more robust and prevents later passes from making decisions using outdated control-flow or definition-use information.

The implemented optimizer includes SSA-style metadata construction, Sparse Conditional Constant Propagation, Aggressive Dead Code Elimination, Loop-Invariant Code Motion, CFG simplification, induction-variable optimization, local GVN/CSE, and interval-based range analysis. The passes are exposed through individual command-line flags as well as a combined `-opt-all` option. The implementation is intentionally conservative: transformations skip ambiguous cases rather than risking semantic changes. This design choice is especially important for loop optimizations, branch rewrites, and value-numbering-style transformations.

The final system demonstrates how standard compiler optimization techniques can be adapted to a compact educational IR. By combining dataflow analysis, CFG reasoning, SSA-style metadata, loop analysis, value numbering, and abstract interpretation, the project turns the baseline Chiron framework into a substantially richer optimization framework while preserving the original program semantics.

2 Chiron IR and Baseline Framework

The project is built on top of the baseline Chiron framework provided with the course. The original framework already supported parsing Chiron source programs, generating an intermediate representation, pretty-printing the IR, constructing a control-flow graph, and running existing analysis hooks such as abstract interpretation and data-flow analysis. Our work extends this base framework by adding a richer optimization layer over the generated IR.

Chiron programs are written in a small turtle-style language, where commands such as **forward**, **right**, assignments, conditionals, and **repeat** loops describe the program behavior. After parsing, the framework lowers the source program into an intermediate representation consisting of command objects paired with relative jump offsets. Each IR instruction is printed with a label such as L0, L1, etc., and the number after the instruction represents the relative jump target or fall-through behavior. For example, a branch condition may jump over part of the IR depending on whether it evaluates to true or false.

This IR representation is close enough to the original source language to be readable, but structured enough to support compiler optimizations. Assignments can be analyzed for constant propagation, expression equivalence, loop invariance, induction-variable patterns, and range information. Conditional commands and relative jumps allow the construction of a control-flow graph, which is essential for optimizations such as SCCP, ADCE, CFG simplification, LICM, and interval analysis.

The control-flow graph divides the IR into basic blocks and records possible control-flow transitions between them. This makes it possible to reason about reachability, loops, branch targets, predecessors, successors, and join points. Several passes in our optimizer rely on this CFG structure. For example, SCCP uses CFG reachability to identify executable blocks, ADCE uses control-dependence information derived from the CFG, LICM identifies loops using backedges, and CFG simplification uses successor and predecessor relationships to safely remove unreachable or trivial blocks.

A key property of the Chiron IR is that control flow is represented using relative offsets. This makes transformations delicate: if a pass inserts, removes, or reorders instructions, jump offsets can become invalid unless they are preserved or recomputed carefully. For this reason, many transformations in our implementation rebuild a fresh IR list and recompute condition

offsets rather than performing fragile in-place edits. Similarly, after a pass rewrites the IR, later analyses rebuild the CFG and SSA metadata to avoid using stale information.

The baseline framework already contained hooks for data-flow analysis and abstract interpretation, but it did not provide the full optimization pipeline implemented in this project. Our contribution is therefore not a replacement for the original framework, but an extension of it. We keep the original parsing, IR generation, CFG construction, and execution model, and add a sequence of optimization passes that operate over the generated IR. The final optimizer can be invoked through individual flags for each pass or through a combined `-opt-all` option.

Overall, the Chiron IR provides a compact but useful setting for implementing classical compiler techniques. It includes variables, arithmetic expressions, branches, loops, and observable turtle commands, while remaining small enough for careful experimentation. This made it possible to implement a broad optimization pipeline while still preserving the semantics of the original Chiron programs.

3 Optimizer Architecture and Pipeline

The optimizer is implemented as a driver-level extension to the Chiron data-flow analysis path. The baseline framework generates an IR sequence and optionally builds a CFG; our implementation uses this as the starting point for a configurable set of optimization passes. Each pass is exposed through a command-line flag, and the driver combines these flags to decide which analyses and transformations should be applied to the current IR.

The architecture follows a staged pipeline rather than a monolithic optimizer. Each pass has a specific responsibility: SCCP reasons about constants and executable control-flow paths, CFG simplification removes structural artifacts exposed by branch rewrites, LICM handles safe loop-invariant assignments, induction-variable optimization performs a restricted strength-reduction transformation, GVN/CSE removes repeated local expressions, interval analysis proves some range-based branch outcomes, and ADCE performs final liveness-based cleanup. This separation made the implementation easier to test because each pass could be run independently before being composed with the others.

The driver maintains the current IR as the shared representation between passes. A transformation may return the same IR unchanged or a rewritten IR containing inserted instructions, removed blocks, constant conditions, or NOP placeholders. Since the CFG and SSA metadata are derived from the IR, the driver rebuilds them before passes that require current control-flow or definition-use information. This is especially important after transformations that change jump offsets, remove branch targets, hoist instructions, or introduce new assignments.

The final `-opt-all` pipeline is arranged so that simplifying passes expose opportunities for later analyses. Constant propagation and interval analysis can turn conditional expressions into constant markers. CFG simplification then removes unreachable or redundant control-flow structure. Loop-oriented passes reduce repeated work inside canonical loops, while value-based passes remove repeated local computations. ADCE is placed near the end so that it can remove assignments made unnecessary by earlier transformations.

Some pass combinations require additional cleanup beyond a single linear ordering. For interval analysis, the optimizer uses a small bounded interaction with CFG simplification. An interval rewrite may convert a condition to `True` or `False`; after CFG simplification removes the dead branch, a later condition may become provable by interval analysis. The implementation therefore performs at most two interval-and-CFG-simplification rounds when both passes are enabled. This gives useful cascading cleanup without turning the whole optimizer into an uncontrolled fixed-point loop.

Individual flags remain useful even though `-opt-all` is the main final pipeline. They allow a single pass to be demonstrated and tested in isolation. For example, `-opt-gvn` shows local common-subexpression elimination without depending on interval analysis, while `-opt-interval`

shows only range-based condition rewriting without deleting the branch structure unless CFG simplification is also enabled. This separation helped validate each pass and also makes the final system easier to explain.

The overall architecture therefore balances modularity and composition. Passes are written as mostly independent analyses or transformations, while the driver coordinates their order, rebuilds derived metadata when needed, and invokes cleanup passes where they are most effective. This design keeps the optimizer understandable while still allowing multiple classical compiler techniques to interact in the final IR.

4 Core Infrastructure

The optimization passes share a common infrastructure rather than each pass maintaining its own independent view of the program. The three central components are the control-flow graph, SSA-style metadata, and careful rebuilding of derived analysis state after IR transformations. These components allow different optimizations to reason about the same Chiron IR consistently while still keeping the individual passes modular.

4.1 Control-Flow Graph

Although Chiron IR is stored as a linear sequence of instructions, conditionals and **repeat** loops introduce non-linear execution paths. The control-flow graph converts this linear representation into a graph of basic blocks connected by control-flow edges. This graph representation is essential because most non-trivial optimizations need more information than the textual order of instructions.

In our optimizer, the CFG is used for several purposes:

- **Reachability:** SCCP and CFG simplification use executable and unreachable blocks to eliminate dead control-flow regions.
- **Loop structure:** LICM and induction-variable optimization identify loops using backedges and loop headers.
- **Control dependence:** ADCE uses CFG-based reasoning to determine which computations are required for observable behavior.
- **Branch cleanup:** CFG simplification uses predecessor and successor relationships to remove unreachable branches and trivial connector blocks.
- **Range-based simplification:** interval analysis rewrites provable conditions, after which CFG simplification uses the graph structure to remove the resulting dead branch regions.

For optimization, the framework generally uses a single-instruction block representation. This keeps the mapping between a CFG block and the original IR instruction direct. Such a representation is useful for targeted rewrites, since a pass can replace a specific condition with **True**, turn a dead assignment into **NOP**, or move a particular loop-invariant instruction while still using graph-level information about predecessors, successors, and loops.

4.2 SSA-Style Metadata

Many optimizations require more information than the source-level variable names provide. For example, two occurrences of `:x` may refer to different assignments if `:x` is redefined along the way. Similarly, after a conditional branch, a use of `:x` may refer to a value coming from either the true branch or the false branch. To reason about such cases, the optimizer constructs SSA-style metadata over the CFG.

The implementation does not rewrite the printed Chiron IR into textual SSA form. Instead, it keeps the original IR unchanged and stores SSA information in side tables. This design preserves compatibility with the existing interpreter and pretty-printer, while still allowing optimization passes to reason about distinct versions of the same variable.

At a high level, SSA construction follows the standard structure:

1. collect variables and definition sites from the CFG,
2. compute a reverse postorder traversal from the **START** block,
3. compute dominators and the dominator tree,
4. compute dominance frontiers,
5. place phi-like metadata at join points,
6. perform stack-based variable renaming.

The dominator information identifies which blocks must execute before other blocks. This is required because SSA renaming depends on dominance: a definition can only be the active reaching definition for a use if it dominates that use. The dominance frontier is used to identify join blocks where definitions from multiple paths may meet. These are the points where traditional SSA would insert phi-functions.

In this project, phi-functions are represented as metadata rather than explicit IR instructions. For example, in a program of the form

```
1 if (:c == 1) [  
2   :x = 10  
3 ] else [  
4   :x = 20  
5 ]  
6 forward :x
```

the final use of `:x` may receive the value from either branch. Conceptually, SSA introduces a phi-function at the join:

```
1 x_2 = phi(x_0, x_1)
```

Our implementation records this relationship internally, while the printed IR still remains in normal Chiron form. This provides the benefits of SSA without changing the execution model.

Variable renaming is implemented using a version counter and a stack for each variable. When the renaming traversal enters a block, it creates versions for phi-like definitions in that block. It then records the current top-of-stack version for every variable use in the instruction. If the instruction defines a variable, a new version is created and pushed onto that variable's stack. When processing successors, the current version is recorded as the incoming value for successor phi metadata. After finishing a dominator-tree subtree, versions created in that subtree are popped from the stack.

This process can be summarized as follows:

```
1 rename(block):  
2   create versions for phi definitions  
3   record current versions for instruction uses  
4   create a new version for the instruction definition  
5   record incoming versions for successor phi nodes  
6   recursively rename dominator-tree children  
7   pop versions created in this block
```

The SSA metadata produced by this process includes definition versions, use versions, definition sites, use sites, and phi-like join information. A small but important implementation detail is that phi definitions and normal instruction definitions are stored separately. This avoids ambiguity in blocks where a phi-like value and an actual assignment to the same variable coexist.

Several later optimizations depend on this metadata. SCCP attaches lattice values to variable versions instead of raw variable names. ADCE traces which definitions contribute to observable commands. LICM checks whether an expression depends on values defined inside or outside a loop. Induction-variable optimization uses versioned definitions to identify loop-carried updates. GVN/CSE uses SSA versions in expression keys so that repeated expressions are only considered equivalent when their operands refer to the same reaching definitions.

Thus, SSA metadata acts as the central value-tracking layer of the optimizer. It gives the later passes enough precision to distinguish different definitions of the same source-level variable, while keeping the Chiron IR readable and executable in the original framework.

4.3 Pass Integration and Fixed-Point Cleanup

The optimizer is organized as a staged pipeline rather than as one large optimization routine. Each pass performs a specific analysis or transformation and then returns an updated IR. The driver is responsible for deciding which passes to run, rebuilding derived analysis structures when required, and invoking cleanup passes after transformations that expose simpler control-flow or dead code.

A central issue in the implementation is that most analyses are only valid for the IR on which they were computed. If a pass rewrites a condition, inserts a hoisted instruction, removes a block, or replaces an assignment with NOP, then the previous CFG and SSA metadata may no longer describe the current program. Therefore, the driver rebuilds CFG and SSA information at important boundaries before running later passes. This prevents later optimizations from using stale reachability, dominance, loop, or def-use information.

The main pass pipeline can be viewed as follows:

1	Initial IR
2	-> local constant folding / algebraic simplification
3	-> SCCP
4	-> LICM
5	-> interval-based condition rewriting
6	-> CFG simplification
7	-> induction-variable optimization
8	-> GVN/CSE
9	-> ADCE
10	-> optimized IR

The exact subset of passes depends on the command-line flags. Each optimization can be enabled independently, while `-opt-all` enables the complete pipeline. This modularity was useful during development because each pass could be tested on focused examples before being composed with the others.

Metadata rebuilding. The driver rebuilds CFG and SSA metadata before passes that rely on current control-flow or definition-use information. For example, SCCP, LICM, induction-variable optimization, GVN/CSE, and ADCE all require up-to-date CFG or SSA information. The helper responsible for this rebuild constructs a fresh CFG from the current IR and then builds SSA metadata over that CFG. This ensures that every analysis observes the latest version of the program.

Cleanup after transformation. Several passes intentionally perform a safe local rewrite and leave structural cleanup to another pass. This avoids mixing multiple responsibilities inside

Situation	Risk Without Rebuild	Driver Response
Condition rewritten	stale branch edges	rebuild CFG before later passes
Assignment inserted	stale def-use data	rebuild SSA before value analyses
Block removed	invalid predecessor/successor data	rebuild CFG before cleanup
Loop rewritten	outdated loop structure	rebuild CFG/SSA before loop-dependent passes

Table 1: Why derived analysis state must be rebuilt after IR transformations.

one transformation. For example, SCCP may replace a condition by `True` or `False`, but CFG simplification is responsible for removing the now-unreachable branch. Similarly, LICM and induction-variable optimization may leave behind NOP instructions, which are later removed or bypassed by CFG simplification when safe.

This separation makes the pipeline easier to reason about. Each pass only needs to prove the safety of its own transformation, while cleanup passes handle the resulting simpler IR patterns.

Bounded fixed-point cleanup. The optimizer does not run all passes in an unbounded fixed-point loop. Such a design would be harder to debug and could make pass interactions unpredictable. Instead, the implementation uses small bounded cleanup loops only where they are clearly useful.

The main example is interval analysis. Interval analysis may prove that a condition is always true or false. Once that branch is simplified by CFG simplification, a later condition may become provable because one branch assignment has been removed. To handle this, the driver performs a bounded interval-and-CFG-simplification cascade when both passes are enabled:

```

1 for at most two rounds:
2   run interval condition rewrite
3   run CFG simplification
4   stop early if the IR no longer changes

```

This gives the benefit of limited fixed-point cleanup without turning the entire optimizer into an uncontrolled repeated pipeline.

IR signatures and change detection. For cleanup loops and post-pass integration, the driver compares a compact signature of the IR before and after a pass. This signature records the instruction type, printed instruction form, and jump target. If the signature does not change, the driver can stop early or avoid running unnecessary cleanup. This is used, for example, to rerun CFG simplification after induction-variable optimization only when the induction-variable pass actually rewrites the IR.

Design summary. The pass integration strategy has three main principles:

- **Passes remain modular:** each pass has a focused responsibility and can be tested independently.
- **Derived metadata is rebuilt after rewrites:** CFG and SSA information are not reused after the IR changes.
- **Cleanup is bounded:** small fixed-point-style cleanup is used only where it gives clear benefit, such as interval analysis followed by CFG simplification.

This integration strategy allows the optimizer to combine many different analyses while keeping the behavior of the full `-opt-all` pipeline predictable and safe.

4.4 Pass Integration and Fixed-Point Cleanup

The optimizer is organized as a staged pipeline rather than as one large optimization routine. Each pass performs a specific analysis or transformation and then returns an updated IR. The driver is responsible for deciding which passes to run, rebuilding derived analysis structures when required, and invoking cleanup passes after transformations that expose simpler control-flow or dead code.

A central issue in the implementation is that most analyses are only valid for the IR on which they were computed. If a pass rewrites a condition, inserts a hoisted instruction, removes a block, or replaces an assignment with `NOP`, then the previous CFG and SSA metadata may no longer describe the current program. Therefore, the driver rebuilds CFG and SSA information at important boundaries before running later passes. This prevents later optimizations from using stale reachability, dominance, loop, or def-use information.

The main pass pipeline can be viewed as follows:

```
1 Initial IR
2   -> local constant folding / algebraic simplification
3   -> SCCP
4   -> LICM
5   -> interval-based condition rewriting
6   -> CFG simplification
7   -> induction-variable optimization
8   -> GVN/CSE
9   -> ADCE
10  -> optimized IR
```

The exact subset of passes depends on the command-line flags. Each optimization can be enabled independently, while `-opt-all` enables the complete pipeline. This modularity was useful during development because each pass could be tested on focused examples before being composed with the others.

Metadata rebuilding. The driver rebuilds CFG and SSA metadata before passes that rely on current control-flow or definition-use information. For example, SCCP, LICM, induction-variable optimization, GVN/CSE, and ADCE all require up-to-date CFG or SSA information. The helper responsible for this rebuild constructs a fresh CFG from the current IR and then builds SSA metadata over that CFG. This ensures that every analysis observes the latest version of the program.

Situation	Risk Without Rebuild	Driver Response
Condition rewritten	stale branch edges	rebuild CFG before later passes
Assignment inserted	stale def-use data	rebuild SSA before value analyses
Block removed	invalid predecessor/successor data	rebuild CFG before cleanup
Loop rewritten	outdated loop structure	rebuild CFG/SSA before loop-dependent passes

Table 2: Why derived analysis state must be rebuilt after IR transformations.

Cleanup after transformation. Several passes intentionally perform a safe local rewrite and leave structural cleanup to another pass. This avoids mixing multiple responsibilities inside one transformation. For example, SCCP may replace a condition by `True` or `False`, but CFG simplification is responsible for removing the now-unreachable branch. Similarly, LICM and induction-variable optimization may leave behind `NOP` instructions, which are later removed or bypassed by CFG simplification when safe.

This separation makes the pipeline easier to reason about. Each pass only needs to prove the safety of its own transformation, while cleanup passes handle the resulting simpler IR patterns.

Bounded fixed-point cleanup. The optimizer does not run all passes in an unbounded fixed-point loop. Such a design would be harder to debug and could make pass interactions unpredictable. Instead, the implementation uses small bounded cleanup loops only where they are clearly useful.

The main example is interval analysis. Interval analysis may prove that a condition is always true or false. Once that branch is simplified by CFG simplification, a later condition may become provable because one branch assignment has been removed. To handle this, the driver performs a bounded interval-and-CFG-simplification cascade when both passes are enabled:

```

1 for at most two rounds:
2   run interval condition rewrite
3   run CFG simplification
4   stop early if the IR no longer changes

```

This gives the benefit of limited fixed-point cleanup without turning the entire optimizer into an uncontrolled repeated pipeline.

IR signatures and change detection. For cleanup loops and post-pass integration, the driver compares a compact signature of the IR before and after a pass. This signature records the instruction type, printed instruction form, and jump target. If the signature does not change, the driver can stop early or avoid running unnecessary cleanup. This is used, for example, to rerun CFG simplification after induction-variable optimization only when the induction-variable pass actually rewrites the IR.

Design summary. The pass integration strategy has three main principles:

- **Passes remain modular:** each pass has a focused responsibility and can be tested independently.
- **Derived metadata is rebuilt after rewrites:** CFG and SSA information are not reused after the IR changes.
- **Cleanup is bounded:** small fixed-point-style cleanup is used only where it gives clear benefit, such as interval analysis followed by CFG simplification.

This integration strategy allows the optimizer to combine many different analyses while keeping the behavior of the full `-opt-all` pipeline predictable and safe.

4.5 Sparse Conditional Constant Propagation

Sparse Conditional Constant Propagation (SCCP) combines constant propagation with reachability analysis over the control-flow graph. Unlike simple local constant folding, SCCP does not only simplify expressions within one instruction. It tracks constant values through SSA versions and simultaneously determines which CFG edges are executable. This allows it to discover both constant expressions and unreachable branches.

The analysis uses a three-level lattice for each SSA variable version:

Lattice value	Meaning
UNDEF	value not known yet
Const(c)	value is known to be the constant c
OVERDEF	value is not a single compile-time constant

The meet operation is designed so that **UNDEF** represents optimistic lack of information, **Const(c)** represents a known constant, and conflicting constants collapse to **OVERDEF**. Thus, if two executable paths provide the same constant value, the result remains constant; if they provide different values, the result becomes overdefined.

SCCP is implemented over SSA metadata rather than raw variable names. Each lattice value is associated with a pair consisting of a variable and an SSA version. This is important because the same source-level variable may have multiple definitions in different parts of the CFG. Tracking versions separately prevents unrelated assignments to the same variable name from being merged prematurely.

The algorithm maintains two worklists:

- a CFG worklist containing edges that have become executable,
- an SSA worklist containing variable versions whose lattice values have changed.

When a CFG edge becomes executable, the destination block becomes reachable and its instruction is evaluated. When an SSA value changes, all blocks using that value are re-evaluated. This interaction is what makes SCCP more precise than running constant propagation and reachability analysis separately.

For assignment commands, SCCP evaluates the right-hand side expression using the current lattice values of its operands. If all operands are constant, the result becomes a constant. If an operand is unknown or overdefined, the result is handled according to the lattice rules. For conditional commands, SCCP evaluates the condition. If the condition is known to be true, only the true successor edge is marked executable; if it is known to be false, only the false successor edge is marked executable. If the condition is overdefined, both successor edges are considered executable.

Phi-like metadata is evaluated using only executable incoming edges. This is a key feature of SCCP. At a join block, values from unreachable predecessors should not affect the lattice value of the phi result. Therefore, SCCP meets only the values coming from predecessor blocks whose incoming edges have been marked executable. This allows it to preserve constants in cases where a normal dataflow analysis would lose precision by merging all syntactic predecessors.

A simplified view of the SCCP algorithm is:

```

1 initialize all SSA definitions to UNDEF
2 mark START executable
3
4 while CFG worklist or SSA worklist is non-empty:
5     process executable CFG edges
6     evaluate newly reachable blocks
7     evaluate phi metadata using executable predecessors
8
9     process changed SSA values
10    re-evaluate blocks that use changed values

```

After the analysis reaches a fixed point, a separate rewrite step applies the discovered constants to the IR. Assignment expressions, movement commands, goto commands, and branch conditions are rewritten when their operands or results are known. If a condition is proven constant, it is replaced by a **True** or **False** condition marker. CFG simplification and ADCE can then remove the unreachable branch structure and any computations that become useless.

For example, consider:

```

1 :x = 5
2 if (:x > 3) [
3     forward 100
4 ] else [

```

```

5   forward 200
6 ]

```

SCCP determines that `:x` is the constant 5, so the condition `:x > 3` is always true. The condition can therefore be rewritten to `True`. After CFG simplification, the false branch is removed, leaving only the executable behavior:

```

1 :x = 5
2 forward 100

```

The SCCP implementation is intentionally conservative. Variables that correspond to external inputs or undefined versions are treated as overdefined, and operations such as division by zero are not folded into unsafe constants. This ensures that SCCP only rewrites expressions and branches when the result is justified by the current SSA and CFG information.

In the overall optimizer, SCCP is one of the earliest global passes. Its main role is to expose later opportunities: constant conditions become targets for CFG simplification, propagated constants create simpler expressions, and unreachable blocks allow ADCE to remove computations that no longer affect observable turtle commands.

4.6 Aggressive Dead Code Elimination

Aggressive Dead Code Elimination (ADCE) removes computations that do not contribute to observable program behavior. In Chiron, observable behavior is mainly produced by turtle commands such as `forward`, `right`, `goto`, pen commands, and pauses. Assignments are not directly observable unless their values are used by such commands or by control-flow decisions that affect such commands. ADCE therefore starts from the observable parts of the program and works backward to find the computations that must be preserved.

The implementation treats an instruction as *essential* if it has a visible side effect. These essential blocks form the initial live set. The algorithm then propagates liveness backward through two kinds of dependence:

- **Data dependence:** if a live instruction uses a variable, the SSA definition producing that variable version must also be live.
- **Control dependence:** if a live block is reachable only because of a branch decision, then the branch condition controlling it must also be live.

This makes ADCE more powerful than a simple local dead-assignment pass. A local pass can remove an assignment only if the assigned variable is never used syntactically later. ADCE instead reasons through SSA def-use information and CFG control dependence, so it can remove computations even in the presence of branches and joins.

The pass first computes post-dominator information over the CFG. Post-dominance is used to derive control dependence: if the execution of one block depends on the outcome of a branch, then that branch must be preserved whenever the dependent block is live. This prevents the optimizer from deleting a branch condition that is necessary to decide whether an observable command executes.

The high-level ADCE algorithm is:

```

1 mark all side-effecting instructions as live
2
3 while live worklist is non-empty:
4     remove a live block from the worklist
5
6     for every variable used in the block:
7         mark the SSA definition of that variable version live

```

```

8      trace phi operands if the definition is phi-like
9
10     for every branch controlling this block:
11         mark the controlling branch live

```

Phi-like metadata requires special handling. If a live value is defined by a phi-like join, then the definitions feeding that phi may also be needed. The implementation therefore recursively traces phi operands back to their defining blocks. This ensures that values flowing through control-flow joins are not incorrectly removed.

After liveness is computed, the driver rewrites dead instructions conservatively. Dead assignments and unreachable side-effect-free instructions can be replaced by NOP. Essential turtle commands are preserved unless they occur in blocks already proven unreachable by earlier analyses such as SCCP. This interaction is important: SCCP may mark a branch unreachable, CFG simplification may remove part of the branch structure, and ADCE then removes assignments that no longer feed any observable command.

For example, consider:

```

1  :x = 10
2  :y = 20
3  :z = :x + :y
4  forward :x

```

The command `forward :x` is essential because it affects the turtle output. Therefore, the definition of `:x` is live. However, `:y` and `:z` do not contribute to any observable command. ADCE can remove them, giving an optimized form such as:

```

1  :x = 10
2  NOP
3  NOP
4  forward :x

```

When combined with constant propagation, the final movement command may be simplified further:

```

1  :x = 10
2  NOP
3  NOP
4  forward 10

```

ADCE is placed near the end of the optimization pipeline because many earlier passes expose dead computations. SCCP may make branches unreachable, LICM and induction-variable optimization may leave behind replaced assignments, GVN/CSE may create local copies that later become unnecessary, and interval analysis may simplify branches. ADCE acts as a final cleanup pass that removes computations no longer needed after these transformations.

The implementation is conservative in its treatment of side effects. Observable turtle operations are treated as roots of liveness, and the pass removes only code that is not required to compute their operands or preserve the control decisions that reach them. This ensures that the optimized IR preserves the visible behavior of the original Chiron program.

4.7 Loop-Invariant Code Motion

Loop-Invariant Code Motion (LICM) reduces redundant computation inside loops. An expression is loop-invariant if its value does not change across loop iterations. If such an expression is computed inside the loop, it can often be moved to a block before the loop so that it is evaluated only once instead of once per iteration.

In Chiron, `repeat` loops are lowered into IR using an internal repeat counter, a loop condition, a loop body, a counter update, and a backedge. This makes LICM a CFG-level optimization

rather than a purely syntactic source-level transformation. The optimizer must identify the loop structure from the CFG, determine which assignments are invariant, and then move them without changing the meaning of the relative jumps.

Loop detection. The implementation detects natural loops using CFG backedges. An edge from a block **tail** to a block **header** is treated as a backedge when the header dominates the tail. Once such a backedge is found, the natural loop is collected by walking backward through predecessors from the tail until the header is reached. This produces the set of blocks belonging to the loop.

For each detected loop, the analysis records:

Loop metadata	Meaning
header	loop entry block dominated by the backedge target
tail	block containing the backedge to the header
blocks	set of blocks belonging to the natural loop
preheader_candidates	predecessors of the header outside the loop
dedicated_preheader	unique safe preheader, when one exists
invariant_candidate_blocks	assignment blocks whose RHS is loop-invariant

The preheader is important because hoisted instructions must be placed on every path into the loop but outside the loop body. The implementation only performs hoisting when it finds a dedicated preheader: a single predecessor outside the loop that falls through directly to the loop header. If the loop entry structure is ambiguous, the pass skips hoisting.

Invariant candidate detection. The pass considers only assignment commands as possible hoist candidates. Internal repeat-counter updates are ignored because they implement loop control and must remain inside the loop. Division expressions are also skipped, because moving them may change when exceptional or undefined behavior occurs, especially if the loop is not executed.

An assignment is considered loop-invariant when all variables used in its right-hand side are defined outside the loop or by assignments already known to be invariant within the same loop. This check uses SSA metadata. For each variable used by the right-hand side, the optimizer looks up the SSA version used at that block and then checks the definition site of that version. If the definition lies outside the loop, the operand is loop-invariant. If it lies inside the loop, it is allowed only if that defining block has already been classified as invariant.

This is computed to a fixed point. Initially, no loop assignments are considered invariant. In each iteration, the pass scans remaining candidate assignments and moves any assignment whose operands are already loop-invariant into the invariant set. This allows chained invariants to be detected. For example:

```

1 repeat 5 [
2     :a = :x + 1
3     :b = :a * 2
4     forward :b
5 ]

```

If `:x` is defined outside the loop, then `:a = :x + 1` is invariant. After that, `:b = :a * 2` also becomes invariant because its operand `:a` is produced by an already-invariant assignment.

Hoisting transformation. After invariant candidates are found, the transformation inserts the hoisted assignments immediately before the loop header, after the dedicated preheader. The original assignment inside the loop is replaced by `NOP`. The pass rebuilds a fresh IR list rather

than editing indices directly. During this rebuild, old condition targets are mapped to new instruction indices so that relative jump offsets remain correct.

A simplified transformation is:

```
1 :b = 200
2 repeat 3 [
3     :c = :b + 1
4     forward :c
5 ]
```

After LICM, the invariant assignment is moved before the loop condition:

```
1 :b = 200
2 :c = :b + 1
3 repeat 3 [
4     NOP
5     forward :c
6 ]
```

In the actual Chiron IR, the loop is represented through explicit counter initialization, condition blocks, and backedges, so the implementation must preserve the relative offsets associated with the loop condition and backedge. This is why the pass rebuilds the IR and recomputes jump offsets from old absolute targets.

Safety conditions. The LICM implementation is deliberately conservative. It hoists only when all of the following conditions hold:

- the loop is a natural loop detected through a valid backedge,
- the loop has a dedicated preheader with an unambiguous insertion point,
- the instruction is an assignment,
- the assignment does not modify the internal repeat counter,
- the right-hand side does not contain division,
- every operand is defined outside the loop or by another invariant assignment.

These restrictions avoid unsafe transformations. For example, moving an expression involving the loop counter would change the value computed in each iteration. Moving an assignment with ambiguous definitions could also change semantics if the value is different along different paths. By skipping such cases, the pass preserves correctness even though it does not hoist every theoretically possible invariant.

Role in the pipeline. LICM is placed before later cleanup and value-based optimizations. Hoisting an invariant computation may leave a NOP in the loop body, which CFG simplification or later cleanup passes can handle. It may also expose new opportunities for constant propagation or GVN/CSE by moving repeated loop computations into a straight-line region before the loop. Thus, LICM contributes both directly, by reducing repeated work inside loops, and indirectly, by simplifying the shape of the IR seen by later passes.

4.8 CFG Simplification

CFG simplification is the structural cleanup pass of the optimizer. Several earlier analyses deliberately leave behind simple but redundant control-flow patterns. For example, SCCP and interval analysis may rewrite a condition to **True** or **False**, ADCE may replace dead assignments with **NOP**, and LICM or induction-variable optimization may move computations out of a loop while leaving a placeholder behind. CFG simplification cleans such artifacts when the control-flow graph proves that doing so is safe.

The pass first collects simplification opportunities from the current CFG. It records unreachable blocks, constant-condition blocks, **NOP**-only blocks, fall-through connector blocks, and straight-line merge candidates. This analysis phase is useful because it separates detection from rewriting: the pass first identifies what could be simplified and then performs only the rewrites whose CFG structure is unambiguous.

Opportunity	Meaning
Unreachable block	block not reachable from START
Constant condition	condition is already True or False
NOP -only block	block contains only no-op instructions
Fall-through connector	reachable NOP block with one flow successor
Redundant marker	constant condition whose taken edge is already fall-through

Constant branch reachability. The first important simplification is constant branch collapse. If a condition has already been rewritten to **True**, only the true successor is reachable. If it has been rewritten to **False**, only the false successor is reachable. The pass computes reachability from **START** while respecting these constant choices. Blocks reachable only through the untaken side of a constant branch are treated as unreachable and can be removed from the rebuilt IR.

This is particularly important after SCCP and interval analysis. These passes do not directly delete branch bodies. Instead, they produce constant condition markers. CFG simplification then interprets those markers at the graph level and removes the dead control-flow region.

For example, after a previous pass rewrites a branch condition, the IR may conceptually contain:

```
1 :x = 5
2 True
3 forward 100
4 right 200
```

The **True** condition makes the false successor unreachable. CFG simplification can remove the dead branch structure and preserve only the executable path.

NOP connector removal. Another common artifact is a **NOP**-only block. Such blocks often appear after dead-code elimination or after a computation has been moved by LICM or replaced by another optimization. Not every **NOP** can be deleted safely, because it may still be a branch target. The pass therefore removes only reachable **NOP**-only blocks that behave as simple fall-through connectors: they must have exactly one flow successor and must not be **START** or **END**.

When a connector block is removed, predecessors are effectively redirected to its successor. The implementation follows chains of removed connector blocks so that a branch does not accidentally target a deleted instruction. If any successor mapping becomes ambiguous, the pass abandons the rewrite and returns the original IR.

Redundant constant markers. After earlier cleanup, some `True` or `False` condition blocks may become redundant markers. A constant marker can be removed only when its single retained edge is equivalent to the next instruction in the rebuilt IR. In other words, the marker is removed only if it no longer changes control flow. The pass also checks that reachable predecessors are local and unambiguous; if a non-local branch target enters the marker, it is preserved.

This conservative rule avoids changing the meaning of branches. A constant marker that is still needed as a jump target or whose taken edge is not simple fall-through is not removed.

IR rebuilding and offset recomputation. The main rewriting step rebuilds a fresh IR from the retained reachable blocks in their original instruction order. During this rebuild, the pass records the new instruction index of each retained block. For condition commands, the false-target offset is recomputed using the new target index. For normal fall-through instructions, the pass validates that the effective successor is exactly the next instruction.

This is necessary because Chiron IR uses relative jump offsets. Removing even one instruction can change the meaning of later jumps if offsets are not updated. CFG simplification therefore treats offset recomputation as part of correctness, not as a formatting detail.

A simplified view of the rebuild is:

1	compute reachable blocks under constant branches
2	choose safe NOP/marker blocks to remove
3	emit retained blocks in IR order
4	map old block positions to new positions
5	recompute condition offsets
6	abort if any successor mapping is ambiguous

Role in the pipeline. CFG simplification is used as a cleanup layer after several other optimizations. SCCP and interval analysis create constant conditions; CFG simplification removes the untaken branches. LICM, ADCE, GVN/CSE, and induction-variable optimization can create NOP blocks or simple fall-through structure; CFG simplification removes or bypasses these where safe. The interval pass also uses a bounded interaction with CFG simplification, because simplifying one range-proven branch can expose another range-proven condition.

The pass is intentionally conservative. It does not attempt arbitrary block merging or aggressive restructuring of the CFG. Instead, it handles the cases that arise frequently from the other implemented passes: unreachable regions, constant branches, NOP connectors, and redundant constant markers. This makes it a reliable structural cleanup pass for the final optimization pipeline.

4.9 Induction-Variable Optimization

Induction-variable optimization targets computations inside loops whose values change in a regular pattern across iterations. A basic induction variable is a loop variable updated by a fixed constant step, such as `:i = :i + 1`. A derived induction variable is computed from a basic induction variable, such as `:d = :i * 4`. Recomputing the derived value by multiplication in every iteration is often unnecessary; it can be replaced by an incremental update.

In this project, induction-variable optimization is implemented as a conservative loop transformation over the Chiron IR. The pass reuses the natural-loop detection infrastructure from LICM, then analyzes assignments inside each loop to identify loop-updated variables, basic induction variables, and derived induction variables.

Basic induction variables. The pass first collects assignments inside each detected loop. Internal Chiron repeat counters, such as variables beginning with `__rep_counter_`, are recorded separately and are not treated as user-level induction variables. A variable is accepted as a

basic induction variable only if it has exactly one non-internal assignment in the loop and the assignment has one of the following forms:

```
1 :i = :i + c
2 :i = c + :i
3 :i = :i - c
```

where `c` is a numeric constant. If a variable has multiple assignments in the loop, or multiple basic-looking updates, it is skipped as ambiguous. This avoids rewriting loops where the variable may not follow a single predictable progression.

Derived induction variables. After basic induction variables are identified, the pass searches for derived induction variables. These are assignments whose right-hand side is a simple expression involving an accepted basic induction variable and a constant. The supported derived forms are:

```
1 :d = :i * c
2 :d = c * :i
3 :d = :i + c
4 :d = c + :i
5 :d = :i - c
```

The transformation phase currently performs strength reduction for the multiplicative case, since it gives the clearest benefit: a multiplication inside the loop can be replaced by an addition. Other derived forms are still detected as analysis information but are not rewritten unless they satisfy the supported rewrite pattern.

Strength reduction. The main rewrite handles a canonical pattern of the form:

```
1 :i = 0
2 repeat 4 [
3     :d = :i * 4
4     forward :d
5     :i = :i + 1
6 ]
```

The original loop recomputes `:d` using multiplication in every iteration. Since `:i` increases by 1 and `:d = :i * 4`, the derived value `:d` increases by 4 each iteration. The optimized form initializes `:d` before the loop, replaces the original multiplication with NOP, and updates `:d` incrementally after the basic induction-variable update:

```
1 :i = 0
2 :d = :i * 4
3 repeat 4 [
4     NOP
5     forward :d
6     :i = :i + 1
7     :d = :d + 4
8 ]
```

In the actual Chiron IR, the loop also contains the internal repeat-counter condition, counter decrement, and backedge. Therefore, the implementation rebuilds the IR and recomputes condition offsets after inserting the initialization and update instructions.

Canonical loop requirements. The rewrite is deliberately restricted to a simple repeat-loop shape. The pass checks that the loop has a positive constant repeat count, a standard internal repeat-counter condition, a standard counter decrement, and a backedge from the loop tail to

the header. It also requires the loop body to be contiguous in the IR and avoids loops containing extra condition commands inside the body.

The strength-reduction rewrite is applied only when all of the following hold:

- the loop has exactly one accepted basic induction variable,
- the loop has exactly one accepted multiplicative derived induction variable,
- neither variable is an internal repeat-counter variable,
- the derived variable depends on the accepted basic induction variable,
- the basic and derived variables each have exactly one assignment in the loop,
- the derived value is used only in positions where the inserted update preserves semantics,
- the derived variable is not used after the loop in a way that would make the rewrite unsafe,
- the loop contains no division expressions.

If more than one rewrite opportunity is found, the pass leaves the IR unchanged rather than applying multiple interacting transformations. This keeps the transformation predictable and avoids subtle interactions between different loop variables.

Safety and interaction with cleanup. The pass skips ambiguous cases instead of trying to infer complex loop behavior. Variables with multiple loop assignments, expressions involving division, multiple changing operands, unsupported nonlinear expressions, and internal repeat-counter updates are not rewritten. These restrictions make the optimization narrower than a full induction-variable framework, but they make it suitable for a small IR with explicit relative jumps.

After rewriting, the driver compares the old and new IR signatures. If the induction-variable pass changed the IR, the driver rebuilds the CFG and runs CFG simplification immediately. This removes simple NOP artifacts left by the transformation and ensures that later passes see a cleaner CFG.

Role in the pipeline. Induction-variable optimization is placed after CFG simplification and interval rewriting, so it operates on a cleaner loop structure. It runs before GVN/CSE and ADCE. This ordering lets later passes simplify copies or dead assignments introduced by the strength-reduction rewrite. In the final pipeline, the pass demonstrates a loop-specific optimization beyond basic constant propagation: it changes the form of a computation so that repeated multiplication inside a loop can be replaced by cheaper incremental addition.

4.10 GVN/CSE

Global Value Numbering (GVN) and Common Subexpression Elimination (CSE) aim to avoid recomputing expressions whose values are already available. In a full compiler, GVN may reason across large regions of the CFG using value numbers and phi-aware equivalence. In this project, we implement a conservative local GVN/CSE pass over straight-line CFG regions. This gives useful common-subexpression elimination while avoiding the complexity and risk of full global value numbering.

The pass first partitions the reachable CFG into straight-line regions. A region is extended through flow edges only when the current block has a single reachable flow successor and that successor has a single reachable predecessor. This restriction ensures that expressions are compared only within unambiguous straight-line control flow. When the pass encounters a control-flow boundary or a non-assignment instruction, it clears the active expression table and begins a new local context.

Expression keys. The central idea is to represent each candidate expression by a canonical key. The pass considers assignment commands whose right-hand sides are simple binary arithmetic expressions using $+$, $-$, or $*$. Division is skipped because it can introduce undefined or exceptional behavior, and unsupported expression forms are ignored.

The expression key contains the operator and the SSA versions of the operands. Using SSA versions is essential. Two expressions that look textually identical are not necessarily equivalent if one of their operands was redefined in between. For example, two occurrences of $:x + 5$ are equivalent only if the use of $:x$ refers to the same SSA version in both expressions.

For commutative operators such as $+$ and $*$, operand keys are sorted so that expressions such as $:x + :y$ and $:y + :x$ receive the same key. For subtraction, the operand order is preserved, so $:x - :y$ and $:y - :x$ are treated as different expressions.

Expression pair	Treated as same?	Reason
$:x + :y, :y + :x$	yes	addition is commutative
$:x * :y, :y * :x$	yes	multiplication is commutative
$:x - :y, :y - :x$	no	subtraction is ordered
$:x / :y, :x / :y$	skipped	division is excluded

Candidate detection. Within each straight-line region, the pass maintains an active table of previously seen expression keys. When an assignment computes a supported expression, the pass checks whether the same key is already active. If not, the assignment becomes the first occurrence of that expression. If the key is already active, the assignment is recorded as a later occurrence of a repeated expression.

Whenever an instruction defines a variable, active expressions depending on that variable are invalidated. This prevents unsafe reuse after an operand has changed. For example:

```

1 :a = :x + 5
2 :x = :x + 1
3 :b = :x + 5

```

The two occurrences of $:x + 5$ should not be considered equivalent, because $:x$ is redefined between them. The invalidation step removes the first expression from the active table when $:x$ is assigned.

Rewrite. For a safe repeated-expression candidate, the first occurrence is kept and later occurrences are rewritten to copy the first result. For example:

```

1 :x = 1
2 :y = 2
3 :a = :x + :y
4 :b = :y + :x
5 :c = :x + :y
6 forward :c

```

The expressions assigned to $:b$ and $:c$ are equivalent to the expression already computed in $:a$. After the CSE rewrite, the program can be represented as:

```

1 :x = 1
2 :y = 2
3 :a = :x + :y
4 :b = :a
5 :c = :a
6 forward :c

```

The pass checks that the first result variable has not been redefined between the first occurrence and a later occurrence. If it has been redefined, the rewrite is skipped. This avoids replacing an expression with a stale value.

Local copy cleanup. The CSE rewrite may introduce simple copy assignments such as `:b = :a`. A second local cleanup step propagates these copies to later uses within the same straight-line region. If the copied variable has no remaining unreplaced uses, the copy assignment is replaced with NOP. Thus the previous example can be further simplified to:

```

1 :x = 1
2 :y = 2
3 :a = :x + :y
4 NOP
5 NOP
6 forward :a

```

This cleanup remains local. It stops at region boundaries and before any redefinition of either the source or destination variable. It also uses SSA use information to avoid deleting a copy if some unreplaced use of its destination version remains.

Safety restrictions. The implementation is intentionally narrower than full global value numbering. It does not reason across arbitrary branches or phi joins, and it does not attempt partial redundancy elimination. It also excludes division and unsupported expression shapes. The pass only rewrites repeated assignment expressions when operand versions, region structure, and redefinition checks all indicate that reuse is safe.

Role in the pipeline. GVN/CSE runs after earlier simplification and loop passes have already cleaned much of the IR. This placement gives it simpler straight-line regions where repeated expressions are easier to identify. Later passes, especially ADCE, can remove NOP instructions and assignments that become unnecessary after CSE. In the final optimizer, GVN/CSE provides a value-based optimization that complements SCCP and interval analysis: even when an expression is not a compile-time constant, it may still be redundant if the same value has already been computed.

4.11 Interval Analysis

Interval analysis is an abstract interpretation technique that tracks a range of possible values for each variable instead of tracking only exact constants. While SCCP can prove that a variable has one known constant value, interval analysis can also reason about variables whose exact value is not known but whose range is bounded. This allows the optimizer to simplify some branches that constant propagation alone cannot prove.

The abstract value used by the analysis is an integer interval:

```

1 [low, high]

```

where either bound may also be infinite. The domain includes a top value, representing the interval `[-inf, +inf]`, and a bottom value, representing an impossible interval. The implementation supports standard interval operations such as join, meet, addition, subtraction, multiplication, division, and unary negation.

Abstract value	Meaning	Example
[5, 5]	exact constant	variable is exactly 5
[3, 7]	bounded range	variable is between 3 and 7
[-inf, +inf]	unknown value	no useful bound is known
_ _	impossible state	unreachable or contradictory state

Forward interval analysis. The pass is implemented as a forward dataflow analysis over the CFG. Each basic block has an input state and one or more output states. A state maps variables to intervals. Assignment commands update the interval of the assigned variable by evaluating the right-hand side expression over interval arithmetic. Non-assignment turtle commands do not change the abstract state.

For example, if the input state contains

1	<code>:x = [3, 7]</code>
	then the assignment
1	<code>:y = :x + 10</code>
	produces
1	<code>:y = [13, 17]</code>

At CFG joins, predecessor states are joined by taking the interval union for each variable. For example, if one branch assigns `:x = 3` and another assigns `:x = 7`, then after the join the analysis records `:x = [3, 7]`. This is less precise than remembering the set `{3, 7}`, but it is conservative: every possible runtime value is still contained in the interval.

Branch narrowing. Condition commands produce separate true and false output states. The analysis narrows these states when the condition gives useful information. For example, if the input state has

1	<code>:x = [0, 20]</code>
	and the condition is
1	<code>:x < 10</code>

then the true branch can narrow `:x` toward values below 10, while the false branch can narrow it toward values at least 10. This branch-sensitive transfer function improves precision inside branch bodies and can help later joins compute tighter intervals.

The implemented narrowing handles common comparison operators such as `<`, `>`, `<=`, `>=`, `==`, and `!=`. For conditions that are too complex to narrow safely, the analysis leaves the states unchanged.

Condition classification. After computing interval states, the optimizer classifies each condition using the input state of its CFG block. A condition can be classified as:

Classification	Meaning
<code>always_true</code>	interval bounds prove the condition must hold
<code>always_false</code>	interval bounds prove the condition cannot hold
<code>unknown</code>	intervals are insufficient to decide the condition

For example, if the input state at a condition contains $:x = [3, 7]$, then the condition $:x < 10$ is classified as `always_true`. If the input state contains $:x = [20, 30]$, then the same condition is classified as `always_false`. If the input state contains $:x = [3, 30]$, then the condition remains `unknown`.

The classification rules are conservative. For a comparison $L < R$, the condition is always true only if the maximum of L is less than the minimum of R . It is always false only if the minimum of L is greater than or equal to the maximum of R . Otherwise, the result is unknown. Similar interval-bound checks are used for the other comparison operators.

Interval-based rewrite. The optimizer uses the condition classification to perform a narrow rewrite. If a condition is classified as `always_true`, it is replaced by a `True` condition marker. If it is classified as `always_false`, it is replaced by a `False` condition marker. The pass does not directly delete branches. Instead, CFG simplification later interprets the constant condition and removes unreachable branch structure.

For example:

```

1  if (:cond == 1) [
2      :x = 3
3  ] else [
4      :x = 7
5  ]
6
7  if (:x < 10) [
8      forward 100
9  ] else [
10     forward 200
11 ]

```

SCCP may not know the exact value of $:x$ after the first branch, because it can be either 3 or 7. Interval analysis, however, computes the range $:x = [3, 7]$. Therefore, the second condition $:x < 10$ is proven always true and can be rewritten to `True`. CFG simplification can then remove the unreachable false branch.

Cascaded cleanup with CFG simplification. Some interval optimizations become possible only after an earlier interval rewrite has been cleaned up by CFG simplification. Consider:

```

1  :x = 10
2  if (:x < 20) [
3      :y = 15
4  ] else [
5      :y = 100
6  ]
7  if (:y < 50) [
8      forward :y
9  ] else [
10     right :y
11 ]

```

Initially, interval analysis can prove that $:x < 20$ is true. However, before CFG simplification removes the false branch, $:y$ may still be approximated as $[15, 100]$, so the later condition $:y < 50$ may remain unknown. After CFG simplification removes the unreachable assignment $:y = 100$, interval analysis can prove that $:y = 15$, and the second condition also becomes true.

To handle this situation, the driver performs a bounded interval-and-CFG-simplification cascade when both passes are enabled:

```

1  for at most two rounds:

```

```

2   run interval condition rewrite
3   run CFG simplification
4   stop if the IR no longer changes

```

This gives useful fixed-point-like behavior without running the entire optimizer repeatedly.

Safety restrictions. The interval rewrite is intentionally narrow. It uses only structured classifications based on the condition block’s input state. It skips existing `True` or `False` markers, ambiguous condition mappings, non-positive jump targets, and any condition involving internal repeat-counter variables. Skipping repeat-counter conditions is important because those conditions implement the semantics of Chiron `repeat` loops, and rewriting them using imprecise interval information could break loop execution.

The analysis is also non-relational. It tracks independent intervals for variables, but it does not track relationships such as $x < y$ or $x = y + 1$ precisely. Therefore, some true facts remain unknown. This is acceptable because the optimizer rewrites only when the interval bounds prove the condition definitely true or definitely false.

Role in the pipeline. Interval analysis complements SCCP. SCCP is stronger when a value is exactly constant, while interval analysis is useful when a value is not known exactly but is still bounded. By rewriting range-provable conditions and then relying on CFG simplification, the interval pass removes branches that are beyond the reach of pure constant propagation. In the final pipeline, it provides an abstract-interpretation-based optimization layer that works together with CFG simplification, ADCE, and later value-based passes.

5 Worked Examples

This section illustrates how the implemented optimizations affect concrete Chiron programs. The examples are intentionally chosen to show both isolated transformations and interactions between multiple passes. Full regression coverage is discussed separately in the evaluation section; here the focus is on understanding how representative optimizations change the IR.

5.1 Full-Pipeline Mixed Example

This example uses a medium-sized Chiron program containing several independent optimization opportunities. The goal is not to make every pass fire on every line, but to demonstrate that different kinds of optimizations can compose under the final `-opt-all` pipeline.

The input program contains patterns for constant propagation, control-flow simplification, repeated expression elimination, interval-based branch simplification, and loop-oriented optimization.

Input program.

```

1 % Paste the full test_full_pipeline_demo.tl program here.

```

Listing 1: Input program for the mixed full-pipeline example.

Command.

```

1 uv run python chiron.py --opt-all ../test_full_pipeline_demo.tl

```

Optimized output.

```

1 % Paste the --opt-all output here.

```

Listing 2: Optimized IR produced by the full pipeline.

Discussion. The example demonstrates the behavior of the optimizer as a composed pipeline rather than as isolated passes. Constant propagation and interval analysis simplify branch conditions when enough information is available. CFG simplification then removes dead or redundant control-flow structure exposed by those rewrites. Loop-oriented passes operate on canonical loop patterns, while GVN/CSE removes repeated local expressions when the same value has already been computed safely. Finally, ADCE removes computations that no longer contribute to observable turtle commands.

The important point is that the final output is not produced by one pass alone. Instead, earlier passes expose opportunities for later passes. This validates the design choice of rebuilding CFG and SSA information between transformations and using cleanup passes after rewrites.

5.2 Interval-Based Cascade Cleanup

This example shows a case where range information enables branch simplification even when exact constant propagation is not sufficient. It also demonstrates why interval analysis is paired with CFG simplification in a bounded cleanup loop.

Input program.

```
1 % Paste test_interval_cascade_cleanup.tl here.
```

Listing 3: Input program for interval-based cleanup.

Interval rewrite only.

```
1 % Paste:
2 % uv run python chiron.py --opt-interval ../test_interval_cascade_cleanup.tl
```

Listing 4: Output with only interval rewriting enabled.

With only `--opt-interval`, the optimizer rewrites conditions that are provably true or false from interval information, but it does not delete branch bodies by itself. This keeps interval rewriting narrow and leaves structural cleanup to CFG simplification.

Interval rewriting with CFG simplification.

```
1 % Paste:
2 % uv run python chiron.py --opt-interval --opt-cfgsimp ../
  test_interval_cascade_cleanup.tl
```

Listing 5: Output with interval rewriting and CFG simplification.

After CFG simplification removes a branch made unreachable by the first interval rewrite, the program becomes simpler. This can also make later range facts more precise.

Full pipeline output.

```
1 % Paste:
2 % uv run python chiron.py --opt-all ../test_interval_cascade_cleanup.tl
```

Listing 6: Output under the complete optimization pipeline.

Discussion. The key idea in this example is that interval analysis can prove conditions from ranges rather than exact constants. For instance, if a variable is known to lie in an interval such as `[3, 7]`, then a condition like `:x < 10` is definitely true even though `:x` is not a single known constant. CFG simplification then removes the unreachable branch created by rewriting the condition to `True` or `False`.

The final `--opt-all` result is usually cleaner than interval rewriting alone because later passes also propagate constants, remove dead computations, and simplify redundant instructions.

5.3 Induction-Variable Strength Reduction

This example demonstrates a loop-specific optimization. The program contains a basic induction variable updated by a constant step and a derived induction variable computed from it. The optimizer replaces repeated multiplication inside the loop with an incremental update.

Input program.

```
1 % Paste test_indvar_derived_mul.tl here.
```

Listing 7: Input program for induction-variable optimization.

Command.

```
1 uv run python chiron.py --opt-indvar ../test_indvar_derived_mul.tl
```

Optimized output.

```
1 % Paste the --opt-indvar output here.
```

Listing 8: Output after induction-variable optimization.

Discussion. The pass identifies a basic induction variable of the form `:i = :i + c` or `:i = :i - c`. It then identifies a derived induction variable such as `:d = :i * k`. If the loop has a simple canonical repeat-loop shape, the derived value can be initialized before the loop and then updated by addition each iteration.

Conceptually, if `:i` increases by 1 and `:d = :i * 4`, then `:d` increases by 4 in every iteration. Therefore, the multiplication inside the loop can be replaced by an addition:

```
1 :d = :d + 4
```

This is a classic strength-reduction transformation. The implementation applies it only when the loop structure, variable updates, and uses of the derived value are unambiguous.

5.4 Local GVN/CSE Example

This example shows how repeated local expressions are eliminated. The pass detects equivalent expressions inside a straight-line CFG region and rewrites later computations to reuse the first result.

Input program.

```
1 % Paste test_gvn_commutative_add_multi.tl or your chosen GVN example here.
```

Listing 9: Input program for local GVN/CSE.

Command.

```
1 uv run python chiron.py --opt-gvn ../test_gvn_commutative_add_multi.tl
```

Optimized output.

```
1 % Paste the --opt-gvn output here.
```

Listing 10: Output after local GVN/CSE.

Discussion. The GVN/CSE pass builds canonical expression keys using the operator and SSA versions of operands. For commutative operators such as addition and multiplication, operands are normalized so that expressions like `:x + :y` and `:y + :x` are treated as the same value when their SSA operand versions match. Later occurrences of the same expression are rewritten to use the first computed result.

A local copy cleanup step then propagates these copies to later uses when safe. If a temporary copy has no remaining unreplaced uses, it can be replaced by `NOP`. This demonstrates that the pass is not only detecting repeated expressions, but also cleaning up the copies introduced by the rewrite.

5.5 Summary of Observed Transformations

The examples above illustrate different categories of optimizations implemented in the project.

Example	Main passes demonstrated	Main effect
Mixed full-pipeline example	multiple passes under <code>-opt-all</code>	verifies pass composition
Interval cascade cleanup	interval analysis + CFG simplification	removes range-provable branches
Induction-variable example	induction-variable optimization	replaces repeated multiplication with
GVN/CSE example	local GVN/CSE	removes repeated local expressions

These examples are complemented by the broader test suite discussed in the next section. The worked examples explain how selected transformations occur, while the evaluation section summarizes the wider set of tests used to validate the implementation.